COPILOTTRAINING
TECHNICAL BRIEFING

GitHub Copilot with Microsoft Foundry

Organizational Memory for Coding Agents

Extend GitHub Copilot agents with organizational memory: the 5–10% of engineering work where intent matters most.

Agentic Systems · CopilotTraining

🤔 THE CORE QUESTION

"When does a GitHub Copilot agent need Microsoft Foundry?"

GitHub gives agents implementation memory. Foundry gives agents **organizational memory.**

Principal Engineer

When does my Copilot agent reach for Foundry instead of staying in GitHub?

Platform Architect

How do I wire organizational knowledge into an agent without rebuilding everything?

GitHub SE

How do I demo something GitHub Copilot alone genuinely cannot do?

5–10%

of coding agent work benefits from organizational memory — the rest stays in GitHub

40%+

relevance improvement with Foundry IQ agentic retrieval vs. traditional RAG

3 layers

of agent context: implementation, operational, and organizational memory

📖 TABLE OF CONTENTS

PART 1

The Two Memories

implementation vs. organizational

The gap bypassFraudChecks() exposes

PART 2

Connecting Foundry IQ

index, expose via MCP, managed auth

Building the organizational memory layer

PART 3

The Organizational Memory Pattern

GitHub MCP + Foundry IQ = one agent

The bypassFraudChecks() question answered

PART 4

Recognizing Intent Problems

the 5–10% sweet spot

When to add Foundry, when to stay in GitHub

PART 1

The Two Memories

What GitHub knows about your code — and the gap only organizational memory fills

Implementation Memory

Code, PRs, issues — what Copilot is built for

Organizational Memory

Architecture decisions, design rationale, intent

The Gap

bypassFraudChecks() — traceable, inexplicable

```
GitHub Copilot answers What? — Foundry IQ answers:
↳ Why?
```

The Code GitHub Can Trace — But Not Explain

CheckoutService.cs

```
if (customer.IsPremium)
{
    bypassFraudChecks();
}
```

✅ **GitHub: PR #413**

Author Alice Mendez, merged 8 months ago, linked to issue #9021

✅ **GitHub: commit history**

Modified 3 times, last touched by the payments team

❓ **GitHub: cannot answer**

Is this bypass intentional? Is the original reason still valid today?

Three Layers of Agent Context



Implementation Context

Code, PRs, issues, tests, CI/CD. GitHub Copilot is optimized for this layer.

- **What exists?**
- **What changed?**
- **How does it work?**



Operational Context

Jira, ServiceNow, monitoring APIs. What is happening right now in the system.

- **What's the status?**
- **Can I take action?**



Intent Context

Architecture reviews, ADRs, decision records. Why the organization built what it built.

- **Why was it built?**
- **What was rejected?**
- **Is the reason still valid?**

GitHub Answers What? — Foundry IQ Answers Why?

💻 GitHub answers: What?

What does this code do?
Code analysis and component understanding

What changed, and when?
Git history, PR timeline, blame

What issue tracks this?
Linked issues and backlogs

What tests cover it?

🧠 Foundry IQ answers: Why?

Why was this designed this way?
Architecture decisions and rationale

What alternatives were considered?
Rejected approaches and tradeoffs

Who approved this exception?
Legal, compliance, or business sign-off

Is the original justification still valid?

Which Context Layer Do We Need?



What exists?

Code analysis, component inventory, dependency graphs → GitHub MCP



What changed?

Git history, PR diffs, deployment records → GitHub MCP



What is happening?

Incidents, open tickets, deployment status → Operational MCPs



Why was it built?

Architecture decisions, rejected alternatives, constraints → Foundry IQ

PART 2

Connecting Foundry IQ

Provisioning organizational memory as an MCP knowledge tool



Index the Knowledge

ADRs, incident reviews, architecture decisions



Expose via MCP

Foundry IQ toolbox — same protocol as GitHub MCP



Managed Identity

No API keys — Entra ID governs access

```
setup-foundry-iq.py → azd deploy → TOOLBOX_ENDPOINT:
↳ organizational memory wired
```

Two Commands to a Running Hosted Agent

azure.yaml

```yaml
services:
  agent:
    host: azure.ai.agent
    source: ./src
    env:
      - key: GITHUB_MCP_ENDPOINT
        value: ${GITHUB_MCP_ENDPOINT}
      - key: TOOLBOX_ENDPOINT
        value: ${TOOLBOX_ENDPOINT}
```

azd up

Provisions Azure resources, builds the container, wires RBAC — one command

GITHUB_MCP_ENDPOINT

GitHub MCP server URL injected at deploy time — implementation memory

TOOLBOX_ENDPOINT

Foundry IQ MCP URL output from setup-foundry-iq.py — organizational memory

High-Signal Sources for Organizational Memory



Architecture Decision Records

Why X was chosen over Y — high signal, low volume. adrs/ folder in the repo.



Incident Postmortems

What failures shaped the current design and the constraints they imposed.



RFC Documents

What alternatives were evaluated and why each was accepted or rejected.



Architecture Review Notes

Design decisions, approvals, compliance constraints at build time.

Managed Identity: Not Auth Plumbing — Governance

🔑 API Key Authentication

Keys stored in config or env
Rotation burden and accidental leak risk

One leaked key exposes the entire index

No identity per agent
All agents share the same credential

Which agents can query what? Unclear.

Managed Identity (Foundry)

Agent's Entra ID governs access
Provisioned automatically at deploy time

No credentials in code or config

Per-agent identity
Each agent has its own access boundary

Which agents earn access? The organization decides.

PART 3

The Organizational Memory Pattern

The bypassFraudChecks() question — answered with organizational memory



The Code

GitHub traces the PR, author, linked issue



The Why

March 2023 architecture review recovered



The Answer

Implementation facts plus organizational intent

```
bypassFraudChecks() → March 2023 Architecture Review →
↳ intent recovered
```

The Architecture Review That Explains bypassFraudChecks()

This is the document Foundry IQ retrieves — the organizational memory GitHub cannot provide

March 2023

Architecture review: the decision document that explains why the code exists

Fraud service was producing 30-second checkout delays for premium customers

⚠️ The business problem

Fraud service latency was causing premium customers to abandon checkout at high rates

⚖️ Legal's decision

Temporary bypass approved pending completion of the fraud platform migration

📅 The constraint

Decision was scheduled for review after migration completion — Q4 2023 target

❓ The open question

The migration completed Q4 2023. Is this exception still required?

💡 GitHub knew the code. The organization knew the reason. Foundry IQ makes both available to the agent.

Two Memory Sources, One Agent

combined-context-agent.py

```
agent = client.agents.create(
  instructions=INSTRUCTIONS,
  tools=[
    McpTool(
      name=GITHUB,
      server_url=GITHUB_MCP_ENDPOINT
    ),
    McpTool(
      name=ORG_MEMORY,
      server_url=TOOLBOX_ENDPOINT
    )
  ]
)
```

💻 github tool

Implementation memory — call first for code-level questions about what and how

🧠 org_memory tool

Organizational memory — call when WHY questions surface that GitHub context cannot answer

📋 INSTRUCTIONS

Use github first. Reach for org_memory when intent questions arise. Surface stale justifications.

The Same Query — Two Different Memory Layers

Why does bypassFraudChecks() exist? Is it still valid?

✗ GitHub Copilot Only

1. **Query received**
   Why does bypassFraudChecks() exist?
2. **GitHub MCP called**
   PR #413, Alice Mendez, issue #9021
3. **Context complete**
   Only implementation facts available
4. **Response returned**
   PR trace only — cannot explain the bypass

✗ **PR #413. Alice Mendez. Cannot explain why the bypass exists.**

✓ GitHub Copilot + Foundry IQ

1. **Query received**
   Why does bypassFraudChecks() exist?
2. **GitHub MCP called**
   PR #413, Alice Mendez, issue #9021
3. **Context incomplete**
   WHY question recognized — calls Foundry IQ
4. **Review retrieved**
   Fraud latency, legal approval, migration status

✓ **Approved March 2023. Migration completed Q4 2023. Review required.**

Same code. Same query. Fundamentally different answer.

From Architecture Archaeology to Answered Questions

❌ The Problem

bypassFraudChecks() exists in production

No one knows why

The engineer who wrote it has left the team

Any refactor could cause a compliance or business incident

💡 The Solution

Combined agent: GitHub MCP + Foundry IQ MCP

March 2023 architecture review retrieved

Fraud latency, legal approval, migration target

✅ The Outcome

Intent recovered in seconds — not days

Validation issue created automatically

Agent proposes: migration completed, is exception still required?

seconds

vs. days of manual archaeology

PART 4

Recognizing Intent Problems

The 5–10% of agent work where GitHub alone is not enough

🔍

Signal Words

Nobody remembers, afraid to touch this

📊

The 5–10% Rule

Most agent work stays in GitHub

🔬

The Filter

If org memory isn't indexed, it won't help

```
Signal: nobody remembers →
↳ organizational memory 5–10% threshold
```

How to Recognize an Intent Problem



Nobody Remembers

Lost implementation rationale — nobody on the current team knows why it was built this way



The Architect Retired

Key institutional context left with a specific person and was never documented



Afraid to Touch This

Unknown constraints producing unknown risk — dangerous legacy code with no paper trail



Migration Keeps Surprising

Historical constraints blocking migration progress that only surface when changes are attempted

The 5–10% Rule: When to Add Foundry IQ

✅ Add Foundry IQ when:

Intent questions appear
Why was this designed this way?

Engineering archaeology risk
Changing code without understanding why

High-value org memory exists
ADRs, RFCs, compliance decisions documented

Team onboarding at scale on complex systems

🎯 Stay in GitHub when:

Code generation and review
90–95% of coding agent work

Refactoring and testing
Implementation-level decisions only

No org memory exists
Indexing noise adds cost without value

Early-stage teams with no decision history

GitHub Copilot Alone vs. GitHub Copilot + Foundry IQ

Before

- Traces bypassFraudChecks() to PR #413
- Identifies author Alice Mendez and commit date
- Links to issue #9021 in the checkout backlog
- Cannot explain why the bypass exists

After

- All GitHub implementation context, plus
- March 2023 architecture review retrieved from Foundry IQ
- Fraud service latency: legal-approved temporary bypass pending migration
- Agent evaluates currency and proposes a validation issue

40%+
relevance improvement with Foundry IQ agentic retrieval vs. traditional RAG

5–10%
of coding agent work where organizational memory changes the answer

seconds
to recover context that would otherwise require days of architecture archaeology

✅ WHAT YOU CAN DO TODAY

Today

- Read both quickstarts: Deploy your first hosted agent + Add Foundry IQ to a hosted agent
- Map your current agent stack against the 3-layer model: implementation, operational, intent
- Identify one signal phrase from the talk that matches a real situation in your codebase

This Week

- Install azd and the Foundry extension: azd ext install microsoft.foundry
- Run azd ai agent init against the basic hosted-agent sample and deploy a working agent
- Identify one knowledge source — ADRs or architecture notes — for a Foundry IQ index

This Month

- Run setup-foundry-iq.py against a real document collection — even 10-20 ADRs
- Wire combined-context-agent.py with GitHub MCP and the Foundry IQ toolbox endpoint
- Test with a real intent question from your codebase — did the agent recover the why?

🔑 Key Takeaway

Start with the recipe: 90-95% of agent work stays in GitHub. Add Foundry when why? questions appear.

REFERENCES

Official Documentation

Quickstart: Deploy your first hosted agent — Microsoft Foundry
azd-based deployment with Entra ID and RBAC

Quickstart: Add a Foundry IQ knowledge base to a hosted agent
Knowledge base provisioning and MCP toolbox wiring

Connect agents to Foundry IQ knowledge bases
Connecting knowledge bases as agent tools with managed identity

Further Reading

Microsoft Foundry Agent Samples
Reference implementations including the Foundry IQ toolbox example (sample 17)

Work IQ overview — Microsoft Copilot Studio
Organizational memory as a service: the M365 intelligence layer

Agentic SDLC — CopilotTraining Tech Talk
Broader context: the full software delivery lifecycle through an agentic lens



GitHub Copilot with Microsoft Foundry

ORGANIZATIONAL MEMORY FOR CODING AGENTS

GitHub answers: What?

bypassFraudChecks() — PR #413, Alice Mendez, issue #9021

Foundry IQ answers: Why?

March 2023 review: fraud latency, legal approval, migration status

5–10% of agent work

Recognize the signals: nobody remembers, afraid to touch this, surprises keep appearing

Ask your Copilot agent: why does this code exist?